

【流浪地球】

题目描述

流浪地球计划在赤道上均匀部署了 N 个转向发动机，按位置顺序编号为 $0 \sim N$

1. 初始状态下所有的发动机都是未启动状态
2. 发动机启动的方式分为“手动启动”和“关联启动”两种方式
3. 如果在时刻 1 一个发动机被启动，下一个时刻 2 与之相邻的两个发动机就会被“关联启动”
4. 如果准备启动某个发动机时，它已经被启动了，则什么都不用做
5. 发动机 0 与发动机 $N-1$ 是相邻的

地球联合政府准备挑选某些发动机在某些时刻进行“手动启动”。当然最终所有的发动机都会被启动。哪些发动机最晚被启动呢？

输入描述

第一行两个数字 N 和 E ，中间有空格

- N 代表部署发动机的总个数， $1 < N \leq 1000$
- E 代表计划手动启动的发动机总个数， $1 \leq E \leq 1000$ ， $E \leq N$

接下来共 E 行，每行都是两个数字 T 和 P ，中间有空格

- T 代表发动机的手动启动时刻， $0 \leq T \leq N$
- P 代表次发动机的位置编号， $0 \leq P < N$

输出描述

第一行一个数字 N ，以回车结束

- N 代表最后被启动的发动机个数

第二行 N 个数字，中间有空格，以回车结束

- 每个数字代表发动机的位置编号，从小到大排序

用例

输入	8 2 0 2 0 6
输出	2 0 4
说明	8个发动机; 时刻0启动 (2,6) ; 时刻1启动 (1,3,5,7) (其中1,3被2关联启动, 5,7被6关联启动) ; 时刻2启动 (0,4) (其中0被1,7关联启动, 4被3,5关联启动) ; 至此所有发动机都被启动, 最后被启动的有2个, 分别是0和4。

题目解析

本题数量级不大，可以考虑暴力破解。思路如下：

假设编号 i 的发动机，手动启动时刻是 **ti**，那么编号 i 的发动机关联启动到编号 j 的发动机，由于

发动机 0 与发动机 N-1 是相邻的，即形成一个环

因此有两种关联方向，我们可以定义为内关联和外关联，其中：

- 内关联 innerDis：需要间隔 **abs(i - j)** 单位时间

- 外关联 outerDis: 需要间隔 $n - \text{abs}(i - j)$ 单位时间

即编号 j 会在 $t_i + \min(\text{innerDis}, \text{outerDis})$ 时刻被关联启动。

按此逻辑，我们可以求出每个发动机，被其余发动机关联启动的时刻，我们只需要保留每个发动机最早启动时刻即可。

我们可以 **定义一个数组** `launches` 记录每个发动机的启动时刻，初始时，假设每个发动机都是在一个极大 1001 时刻启动 ($0 \leq T \leq N \leq 1000$)，这样后续，我们就可以比较保留下较小值。

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      // 发动机的总个数，计划手动启动的发动机总个数
7      const [n, e] = (await readline()).split(" ").map(Number);
8
9      // 记录每个发动机的最终启动时刻，初始化为极大值，方便后面取最早启动时刻
10     const launches = new Array(n).fill(1001);
11     for (let i = 0; i < e; i++) {
12         // 发动机的手动启动时刻，发动机的位置编号
13         const [t, p] = (await readline()).split(" ").map(Number);
14         // p号发动机在t时刻手动启动
15         launches[p] = t;
16     }
17
18     // 从编号 i 的发动机手动启动后，关联启动到编号 j
19     for (let i = 0; i < n; i++) {
20         for (let j = 0; j < n; j++) {
21             // 内关联距离
22             const innerDis = Math.abs(i - j);
23             // 外关联距离
24             const outerDis = n - innerDis;
```

```

25     // 最短关联距离
26     const dis = Math.min(innerDis, outerDis);
27
28     launches[j] = Math.min(launches[j], launches[i] + dis);
29 }
30 }
31
32 let maxT = 0; // 最晚启动时刻
33 const last = []; // 最晚启动的发动机编号集合
34
35 for (let p = 0; p < launches.length; p++) {
36     const t = launches[p]; // 当前发动机启动时刻
37
38     if (t < maxT) continue; // 不是最晚启动的发动机
39
40     // 更晚启动的时刻
41     if (t > maxT) {
42         maxT = t;
43         last.length = 0;
44     }
45
46     last.push(p); // 记录该发动机编号
47 }
48
49 console.log(last.length);
50 console.log(last.sort((a, b) => a - b).join(" "));
51 }());

```

Java算法源码

```

1 import java.util.ArrayList;
2 import java.util.Arrays;
3 import java.util.Scanner;
4 import java.util.StringJoiner;
5

```

```

6 | public class Main { 7 |     public static void main(String[] args) {
8 |         Scanner sc = new Scanner(System.in);
9 |
10 |         int n = sc.nextInt(); // 发动机的总个数
11 |         int e = sc.nextInt(); // 计划手动启动的发动机总个数
12 |
13 |         int[] launches = new int[n]; // 记录每个发动机的最终启动时刻
14 |         Arrays.fill(launches, 1001); // 初始化为极大值，方便后面取最早启动时刻
15 |
16 |         for (int i = 0; i < e; i++) {
17 |             int t = sc.nextInt(); // 发动机的手动启动时刻
18 |             int p = sc.nextInt(); // 发动机的位置编号
19 |
20 |             launches[p] = t; // p号发动机在t时刻手动启动
21 |         }
22 |
23 |         // 从编号 i 的发动机手动启动后，关联启动到编号 j
24 |         for (int i = 0; i < n; i++) {
25 |             for (int j = 0; j < n; j++) {
26 |                 // 内关联距离
27 |                 int innerDis = Math.abs(i - j);
28 |                 // 外关联距离
29 |                 int outerDis = n - innerDis;
30 |                 // 最短关联距离
31 |                 int dis = Math.min(innerDis, outerDis);
32 |                 launches[j] = Math.min(launches[j], launches[i] + dis);
33 |             }
34 |         }
35 |
36 |         int maxT = 0; // 最晚启动时刻
37 |         ArrayList<Integer> last = new ArrayList<>(); // 最晚启动的发动机编号集合
38 |
39 |         for (int p = 0; p < launches.length; p++) {
40 |             int t = launches[p]; // 当前发动机启动时刻
41 |
42 |             if (t < maxT) continue; // 不是最晚启动的发动机
43 |

```

```

44         if (t > maxT) { // 更晚启动的时刻
45                                     maxT = t;
46         last.clear();
47     }
48
49     last.add(p); // 记录该发动机编号
50 }
51
52     StringJoiner sj = new StringJoiner(" ");
53     last.stream().sorted((a, b) -> a - b).forEach(p -> sj.add(p + " "));
54
55     System.out.println(last.size());
56     System.out.println(sj);
57 }
58 }

```

Python算法源码

```

1  if __name__ == '__main__':
2      # 发动机的总个数，计划手动启动的发动机总个数
3      n, e = map(int, input().split())
4
5      # 记录每个发动机的最终启动时刻，初始化为极大值，方便后面取最早启动时刻
6      launches = [1001] * n
7
8      for _ in range(e):
9          # 发动机的手动启动时刻，发动机的位置编号
10         t, p = map(int, input().split())
11         # p号发动机在t时刻手动启动
12         launches[p] = t
13
14     # 从编号 i 的发动机手动启动后，关联启动到编号 j
15     for i in range(n):
16         for j in range(n):
17             # 内关联距离

```

```

18         innerDis = abs(i - j)
19         # 外关联距离
20         outerDis = n - innerDis
21         # 最短关联距离
22         minDis = min(innerDis, outerDis)
23         launches[j] = min(launches[j], launches[i] + minDis)
24
25     maxT = 0 # 最晚启动时刻
26     last = [] # 最晚启动的发动机编号集合
27
28     for p in range(n):
29         t = launches[p] # 当前发动机启动时刻
30
31         if t < maxT: # 不是最晚启动的发动机
32             continue
33
34         # 更晚启动的时刻
35         if t > maxT:
36             maxT = t
37             last.clear()
38
39         last.append(p) # 记录该发动机编号
40
41     last.sort()
42
43     print(len(last))
44     print(" ".join(map(str, last)))

```

C算法源码

```

1 #include <stdio.h>
2 #include <math.h>
3 #include <stdlib.h>
4
5 int cmp(const void *a, const void *b) {

```

```

6 |     return *(int *) a - *(int *) b; 7 | }
8 |
9 | int main() {
10 |     // 发动机的总个数，计划手动启动的发动机总个数
11 |     int n, e;
12 |     scanf("%d %d", &n, &e);
13 |
14 |     // 记录每个发动机的最终启动时刻
15 |     int launches[n];
16 |     for (int i = 0; i < n; i++) {
17 |         launches[i] = 1001; // 初始化为极大值，方便后面取最早启动时刻
18 |     }
19 |
20 |     for (int i = 0; i < e; i++) {
21 |         // 发动机的手动启动时刻，发动机的位置编号
22 |         int t, p;
23 |         scanf("%d %d", &t, &p);
24 |
25 |         launches[p] = t; // p号发动机在t时刻手动启动
26 |     }
27 |
28 |     // 从编号 i 的发动机手动启动后，关联启动到编号 j
29 |     for (int i = 0; i < n; i++) {
30 |         for (int j = 0; j < n; j++) {
31 |             // 内关联距离
32 |             int innerDis = abs(i - j);
33 |             // 外关联距离
34 |             int outerDis = n - innerDis;
35 |             // 最短关联距离
36 |             int dis = (int) fmin(innerDis, outerDis);
37 |             launches[j] = (int) fmin(launches[j], launches[i] + dis);
38 |         }
39 |     }
40 |
41 |     // 最晚启动时刻
42 |     int maxT = 0;

```

```

43 |      // 最晚启动的发动机编号集合
44 |      int last[n];
45 |      int last_size = 0;
46 |
47 |      for (int p = 0; p < n; p++) {
48 |          int t = launches[p]; // 当前发动机启动时刻
49 |
50 |          if (t < maxT) continue; // 不是最晚启动的发动机
51 |
52 |          if (t > maxT) { // 更晚启动的时刻
53 |              maxT = t;
54 |              last_size = 0;
55 |          }
56 |
57 |          last[last_size++] = p; // 记录该发动机编号
58 |      }
59 |
60 |      qsort(last, last_size, sizeof(last[0]), cmp);
61 |
62 |      printf("%d\n", last_size);
63 |
64 |      for (int i = 0; i < last_size; i++) {
65 |          printf("%d ", last[i]);
66 |      }
67 |
68 |      return 0;
69 |
70 | }

```

C++ 算法源码

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |

```

```
5 int main() { 6 | // 发动机的总个数, 计划手动启动的发动机总个数
7     int n, e;
8     cin >> n >> e;
9
10    // 记录每个发动机的最终启动时刻, 初始化为极大值, 方便后面取最早启动时刻
11    vector<int> launches(n, 1001);
12
13    for (int i = 0; i < e; i++) {
14        // 发动机的手动启动时刻, 发动机的位置编号
15        int t, p;
16        cin >> t >> p;
17
18        launches[p] = t; // p号发动机在t时刻手动启动
19    }
20
21    // 从编号 i 的发动机手动启动后, 关联启动到编号 j
22    for (int i = 0; i < n; i++) {
23        for (int j = 0; j < n; j++) {
24            // 内关联距离
25            int innerDis = abs(i - j);
26            // 外关联距离
27            int outerDis = n - innerDis;
28            // 最短关联距离
29            int dis = min(innerDis, outerDis);
30            launches[j] = min(launches[j], launches[i] + dis);
31        }
32    }
33
34    // 最晚启动时刻
35    int maxT = 0;
36
37    // 最晚启动的发动机编号集合
38    vector<int> last;
39
40    for (int p = 0; p < n; p++) {
41        int t = launches[p]; // 当前发动机启动时刻
42    }
```

```
43 |         if (t < maxT) continue; // 不是最晚启动的发动机
44 |
45 |         if (t > maxT) { // 更晚启动的时刻
46 |             maxT = t;
47 |             last.clear();
48 |         }
49 |
50 |         last.emplace_back(p); // 记录该发动机编号
51 |     }
52 |
53 |     sort(last.begin(), last.end());
54 |
55 |     cout << last.size() << endl;
56 |     for (const auto &p: last) {
57 |         cout << p << " ";
58 |     }
59 |
60 |     return 0;
61 | }
```